

Joins & data integration

CIS 5450, Fall 2026

Ryan Marcus



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

Outline

- Course logistics
- Brief review
- Other types of joins
- What is entity linking?
- String-label entity linking

Outline

- **Course logistics**
- Brief review
- Other types of joins
- What is entity linking?
- String-label entity linking

Upcoming Deadlines

- HW1 due Sept 16th at 10:00pm (2 days)
- HW2 releasing Sept 16th at 10:00pm (2 days)
- OE2 signups starting soon

HW Grading

- Autograder is a set of *unit tests*
 - necessary but not sufficient condition of correctness
 - All valid programs pass the unit tests
 - But not all programs that pass the unit tests are valid

Question: write code to scrap a Wikipedia page and extract Bill Gates' birthday.

Solution that will pass the autograder but will get 0 points:

```
print("Oct 28th, 1955")
```

Outline

- Course logistics
- **Brief review**
- Other types of joins
- What is entity linking?
- String-label entity linking

Combining Data

- Combining data **that was intended to be combined**
- Combining data that was **not** intended to be combined

Joining

- Pure computation

Data Integration

- More complex
- Requires knowledge

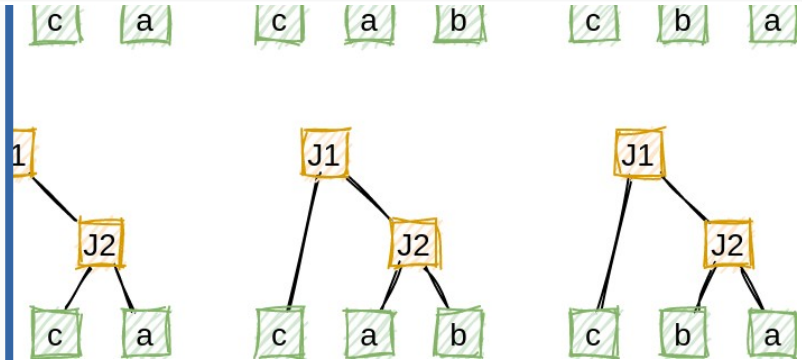
Join Orders

n	a(n)
2	2
3	12
4	120

```
[24] %%time
      sony = df_company_name[df_company_name["name"].str.contains("Sony")]
      sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
      aj = df_name[df_name["name"] == "Jolie, Angelina"]
      aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

      aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
      result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
      result["title"].sort_values().unique()
```

13	1295295050649600
14	64764752532480000
15	3497296636753920000
16	202843204931727360000
17	12576278705767096320000
18	830034394580628357120000
19	581024076206439849984000000



Something feels off?

Strings

Slow, manual code (we hate this):

```
buf = ""
for i in s:
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:
        buf += i
return buf
```

Fast, declarative code (we love this):

`re.sub("[^0-9]", "", s)`

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Replace with empty string

Joins

Slow, manual code (we hate this):

```
[24] %%time
sony = df_company_name[df_company_name["name"].str.contains("Sony")]
sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
aj = df_name[df_name["name"] == "Jolie, Angelina"]
aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
result["title"].sort_values().unique()
```

Fast, declarative code (we love this):



Mental Model of Joins

- A join is a **Cartesian product** then a **filter**

$$\sigma_{\text{equal letters}} (S_1 \times S_2)$$

=

$$S_1 \bowtie_{\text{equal letters}} S_2$$

In the end, we want
distinct title values

SQL

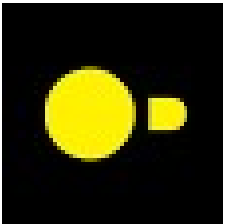
Take the cross product of
all 5 tables

Do all the join filters

Do our base table filters

Sort the data by title

```
con.sql("""  
SELECT DISTINCT title.title  
FROM name, title, cast info, company name, movie companies  
WHERE name.id = cast_info.person_id  
      AND cast_info.movie_id = title.id  
      AND title.id = movie_companies.movie_id  
      AND movie_companies.company_id = company_name.id  
      AND name.name = 'Jolie, Angelina'  
      AND company_name.name LIKE '%Sony%'  
ORDER BY title  
""")
```



DuckDB

Outline

- **Course logistics**
- Brief review
- **Other types of joins**
- What is entity linking?
- String-label entity linking

Inner Join

- Before now: “inner join”
 - Most mathematically well-defined
 - Most common type
- Matching rows of A and B, one row per match

ID	Student Name
08744	Eden
827401	Ringo
172341	Sarah

S_ID	Course
08744	Big Data
08744	Organic Chemistry
172341	Econometrics

ID	Student name	Course
08744	Eden	Big Data
08744	Eden	Organic Chemistry
172341	Sarah	Econometrics

```
SELECT *  
FROM A, B  
WHERE A.a = B.a;
```

Inner Join

- Before now: “inner join”
 - Most mathematically well-defined
 - Most common type
- Matching rows of A and B, one row per match

ID	Student Name
08744	Eden
827401	Ringo
172341	Sarah

S_ID	Course
08744	Big Data
08744	Organic Chemistry
172341	Econometrics

ID	Student name	Course
08744	Eden	Big Data
08744	Eden	Organic Chemistry
172341	Sarah	Econometrics

```
SELECT *  
FROM A, B  
WHERE A.a = B.a;
```

```
SELECT *  
FROM A INNER JOIN B  
ON A.a = B.a  
WHERE ...;
```

(L/R) Outer Join

- Outer join
 - Keeps all rows from one side, matches with “null” if no match
 - Good for getting “empty” sets
- All rows of A with matching rows of B, one row per match

```
SELECT *  
FROM A  
    LEFT JOIN B  
    ON A.a = B.a;
```

ID	Student Name
08744	Eden
827401	Ringo
172341	Sarah

S_ID	Course
08744	Big Data
08744	Organic Chemistry
172341	Econometrics

ID	Student name	Course
08744	Eden	Big Data
08744	Eden	Organic Chem
172341	Sarah	Econometrics
827401	Ringo	<i>NULL</i>

Semi Join

- Semi join
 - Keeps all rows from one side that match
 - Good for answering “any”
- All rows of A that have a match in B, once

```
SELECT * FROM A
WHERE EXISTS (
  SELECT * FROM B
  WHERE A.a = B.b
)
```

ID	Student Name
08744	Eden
827401	Ringo
172341	Sarah

S_ID	Course
08744	Big Data
08744	Organic Chemistry
172341	Econometrics

ID	Student name
08744	Eden
172341	Sarah

Anti Join

- Anti join
 - Keeps all rows from one side that **don't** match
 - Good for answering “none”
- All rows of A that **don't** have a match in B, once

```
SELECT * FROM A
WHERE NOT EXISTS (
  SELECT * FROM B
  WHERE A.a = B.b
)
```

ID	Student Name
08744	Eden
827401	Ringo
172341	Sarah

S_ID	Course
08744	Big Data
08744	Organic Chemistry
172341	Econometrics

ID	Student name
827401	Ringo

Division

- Division
 - Keeps rows in A that match **every** row in B
 - Good for answering “all”
- All rows of A that match all rows B

Hmmm... how can we implement this? Idea:

- 1) Take each distinct student, cross product with required courses
- 2) Subtract that from A
- 3) Then....

Student	Course
Eden	Big Data
Eden	Organic Chemistry
Sarah	Econometrics
Sarah	Big Data
Sarah	Intro to Music
Ringo	Econometrics
Ringo	Big Data

Course
Big Data
Econometrics

Student
Sarah
Ringo

Inner Join



Each matching row between A and B
One row per match

Outer Join



All rows of A with matching rows of B
One row per match, NULL for non-match

Semi Join



Rows of A with **any** match in B
One row per row of A

Anti Join



Rows of A with **no** match in B
One row per row of A

Division



Rows of A that match **all** rows in B
One row per row of A

Outline

- Course logistics
- Brief review
- Other types of joins
- **String-label entity linking**

Entity Linking

- Simplest form of data integration
 - Matching the rows of one relation with the rows of another that isn't a “normal” join

Entity Linking

- Simplest form of data integration
 - Matching the rows of one relation with the rows of another that isn't a "normal" join

Dept	Admin
CIS	Cheryl
Comm	Andy

Dept	Prof
Computer & Information Science	Marcus
Communications	Weinberg

String-label Entity Linking

- We want to link “CIS” with “Computer & Information Science”
- ... but couldn't CIS be **C**ommun**i**cation**S**?
- Will have to settle for something imprecise

Dept	Admin
CIS	Cheryl
Comm	Andy

Dept	Prof
Computer & Information Science	Marcus
Communications	Weinberg

Scoring Functions

- Ideally, we want a function f s.t.:
 - $f(\text{"cis"}, \text{"computer \& information science"}) == \text{True}$
 - $f(\text{"comm"}, \text{"computer \& information science"}) == \text{False}$
 - $f(\text{"cis"}, \text{"communications"}) == \text{False}$
 - $f(\text{"comm"}, \text{"communications"}) == \text{True}$
- ... but such a function doesn't really exist.
- So instead, we'll settle for a *scoring* function s :
 - $s(\text{"cis"}, \text{"computer \& information science"}) == 0.8$
 - $s(\text{"cis"}, \text{"communications"}) == 0.3$

Edit Distance

computer & information science
computer & **i**nformation **s**cience

Insert "omputer &"
Insert "nformation"
Insert "cience"

communications
communi**i**cations

Insert "ommun"
Insert "cation"

computer & information science
computer & **infor**mation science

Insert "puter & infor"
Insert "ation science"

communications
Communications

Insert "unications"

Edit Distance

```
[1] from difflib import SequenceMatcher
```

```
def s(a, b):  
    return SequenceMatcher(None, a, b).ratio()
```

1 = identical
0 = nothing in common



Ut oh... "comm" is scored
as a better match!



```
s("cis", "computer & information science")
```

0.12121212121212122

```
s("comm", "computer & information science")
```

0.23529411764705882

Edit Distance

Comm

CIS

computer &
information
science

communications

Edit Distance

Comm

computer &
information
science

CIS

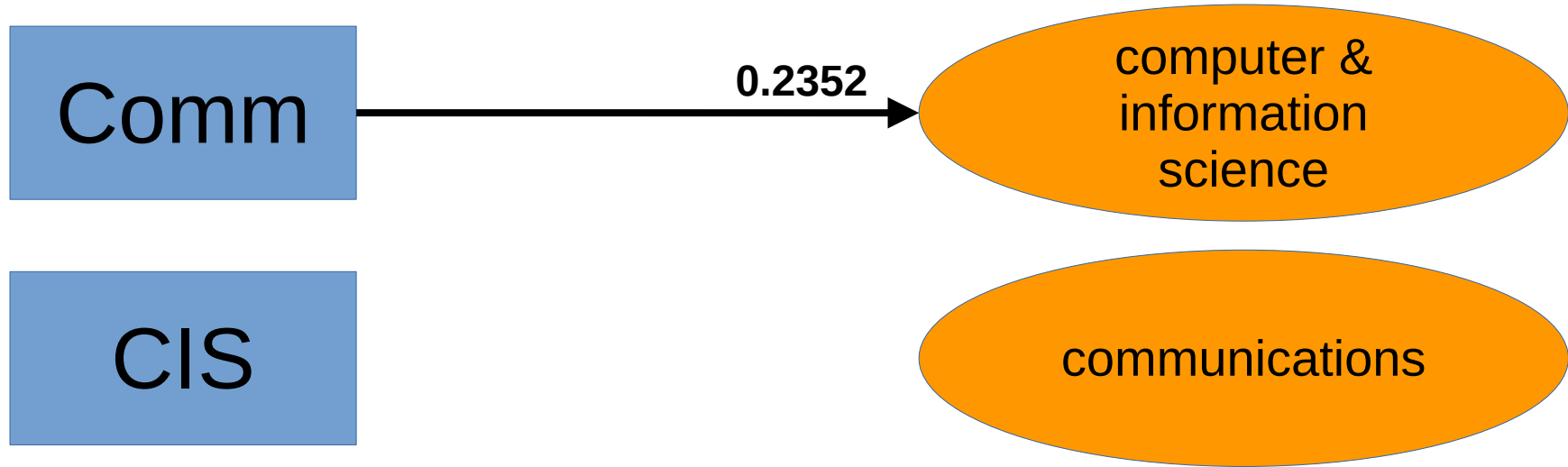
communications

```
[34] s("Comm", "Computer & Information Science")
```



0.23529411764705882

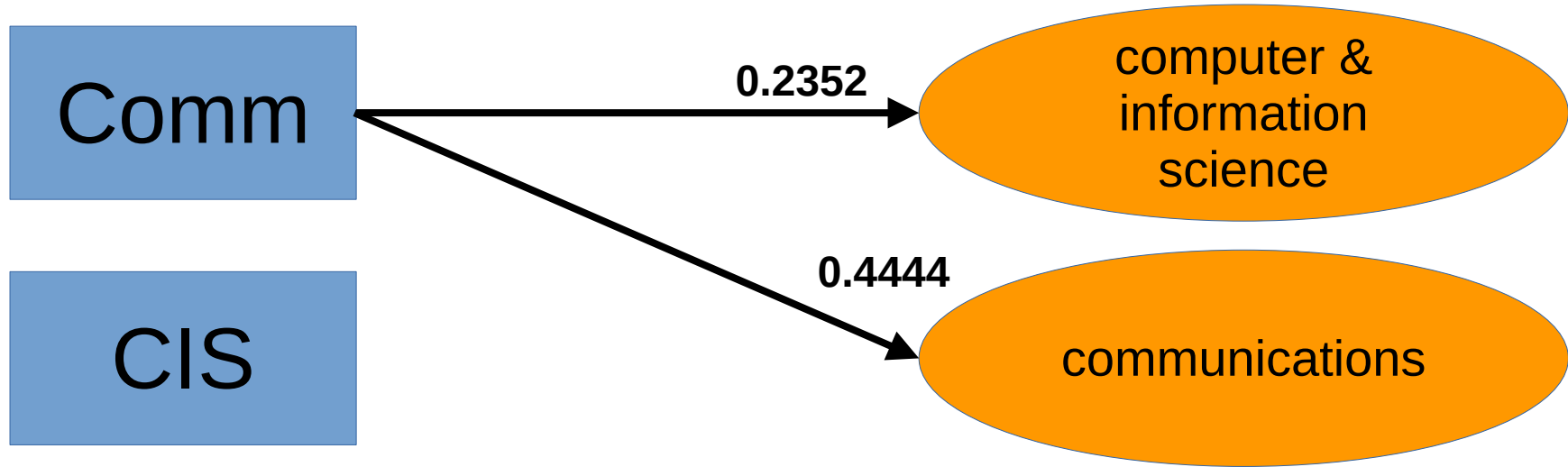
Edit Distance



```
[34] s("Comm", "Computer & Information Science")
```

```
→ 0.23529411764705882
```

Edit Distance

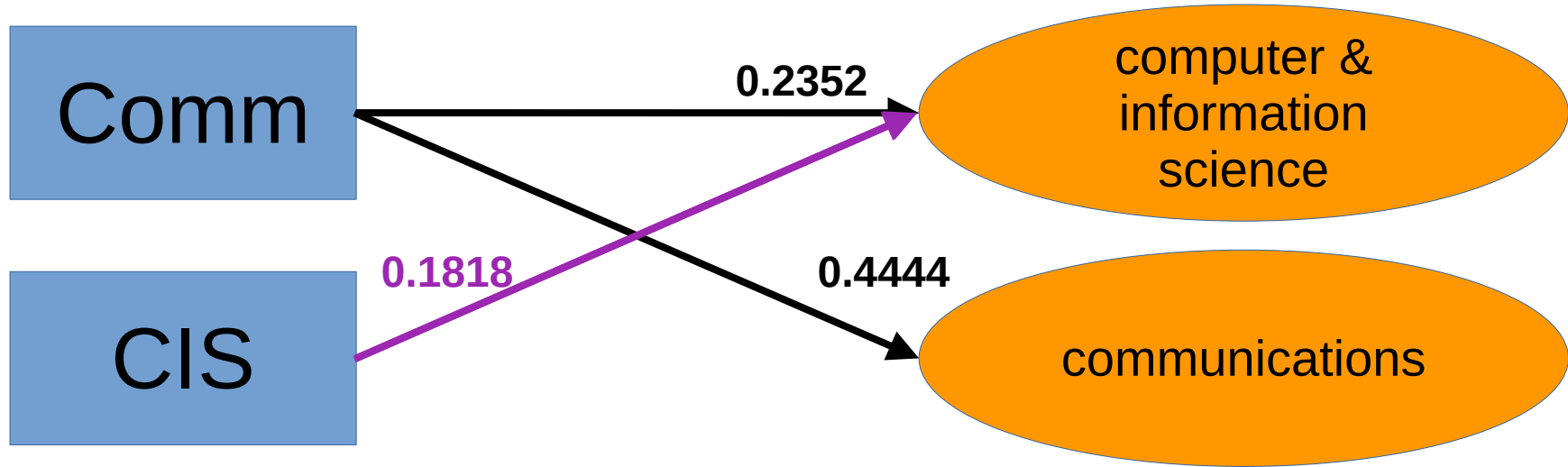


```
[35] s("Comm", "Communications")
```



```
0.444444444444444444
```

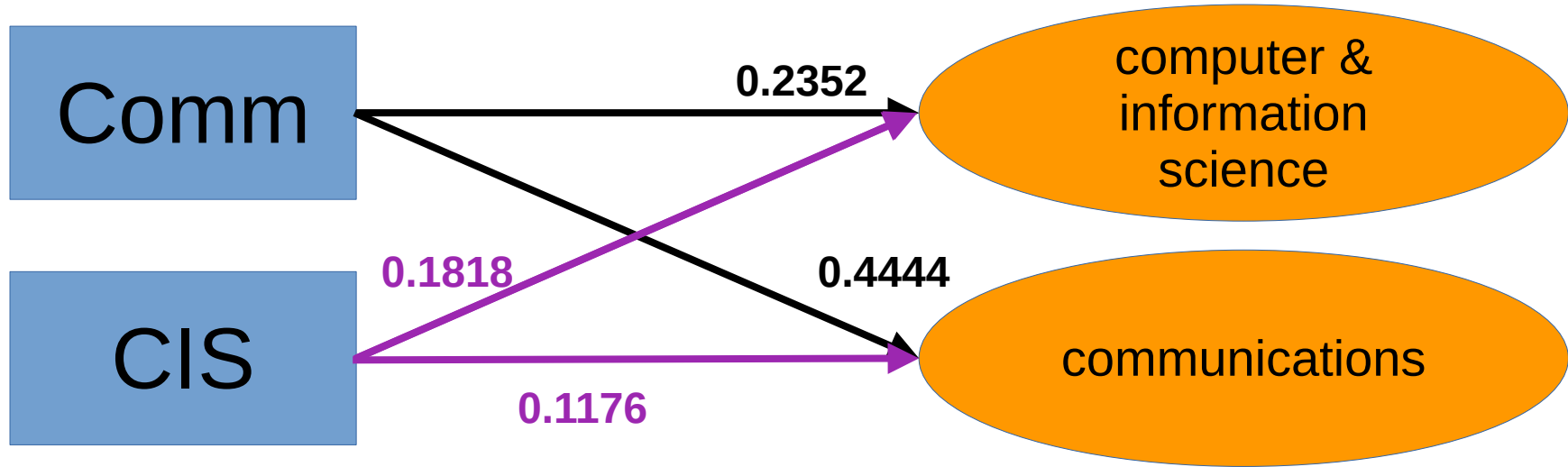
Edit Distance



```
[32] s("CIS", "Computer & Information Science")
```

```
0.18181818181818182
```

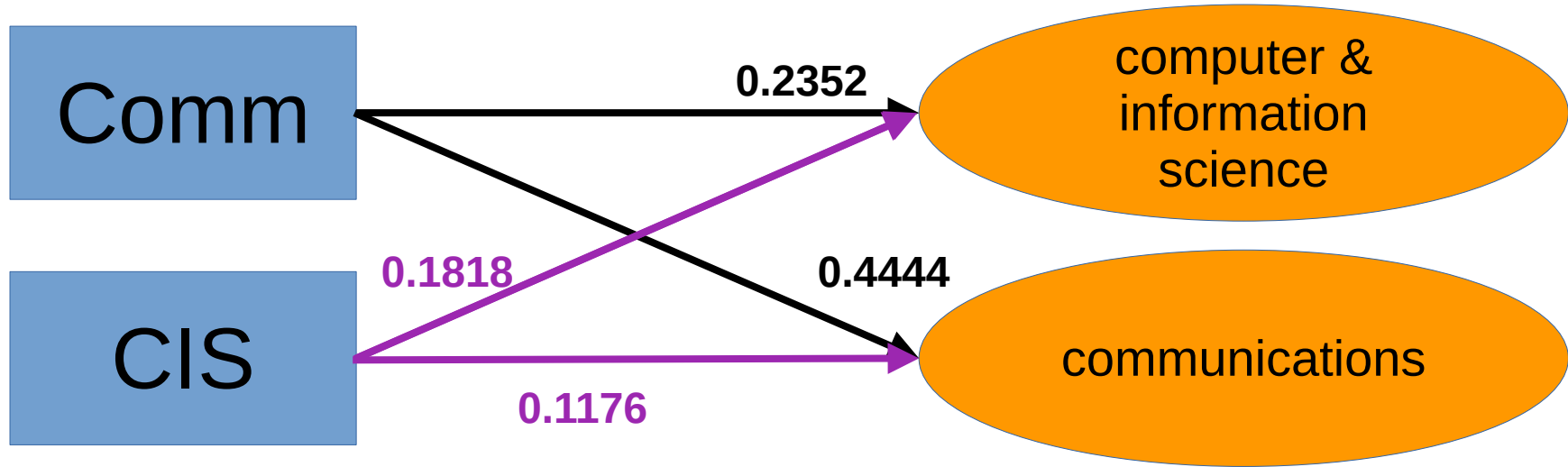
Edit Distance



```
[33] s("CIS", "Communications")
```

0.11764705882352941

Edit Distance



Problem: “comm” is a better match for both!

Edit Distance

- “Comm” is a better match for both departments

Dept	Admin
CIS	Cheryl
Comm	Andy

Dept	Prof
Computer & Information Science	Marcus
Computer & Information Science	Gardner
Computer & Information Science	Davidson
Communications	Weinberg

Edit Distance

- “Comm” is a better match for both departments

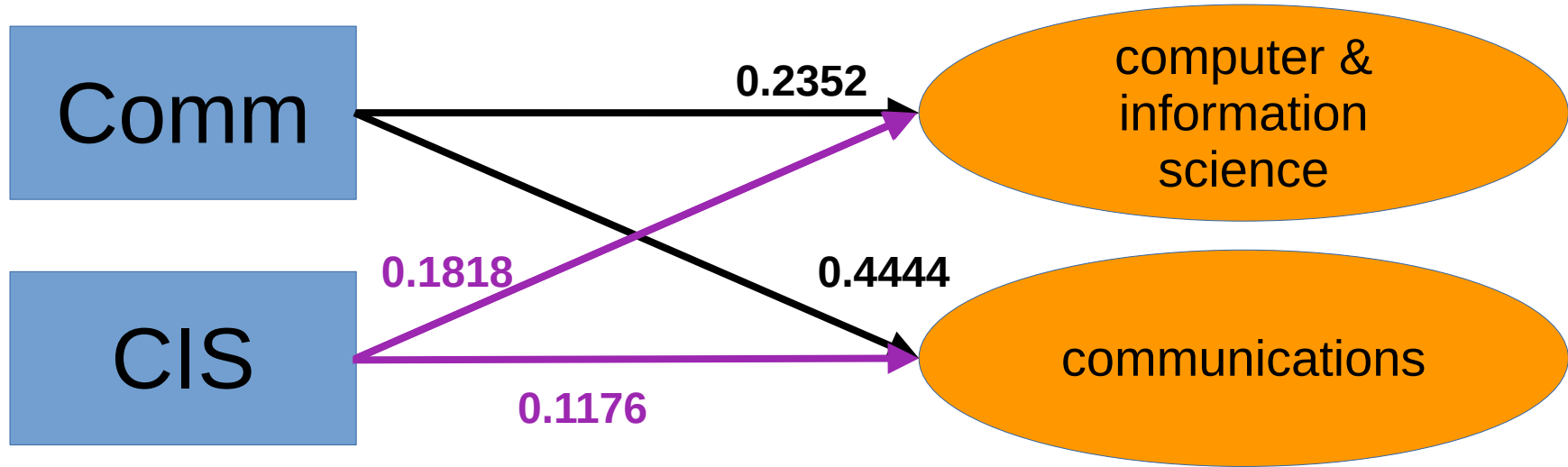
Dept	Admin
CIS	Cheryl
Comm	Andy

Dept	Prof
Computer & Information Science	Marcus
Computer & Information Science	Gardner
Computer & Information Science	Davidson
Communications	Weinberg

One of these should
match with one **and**
only one of these

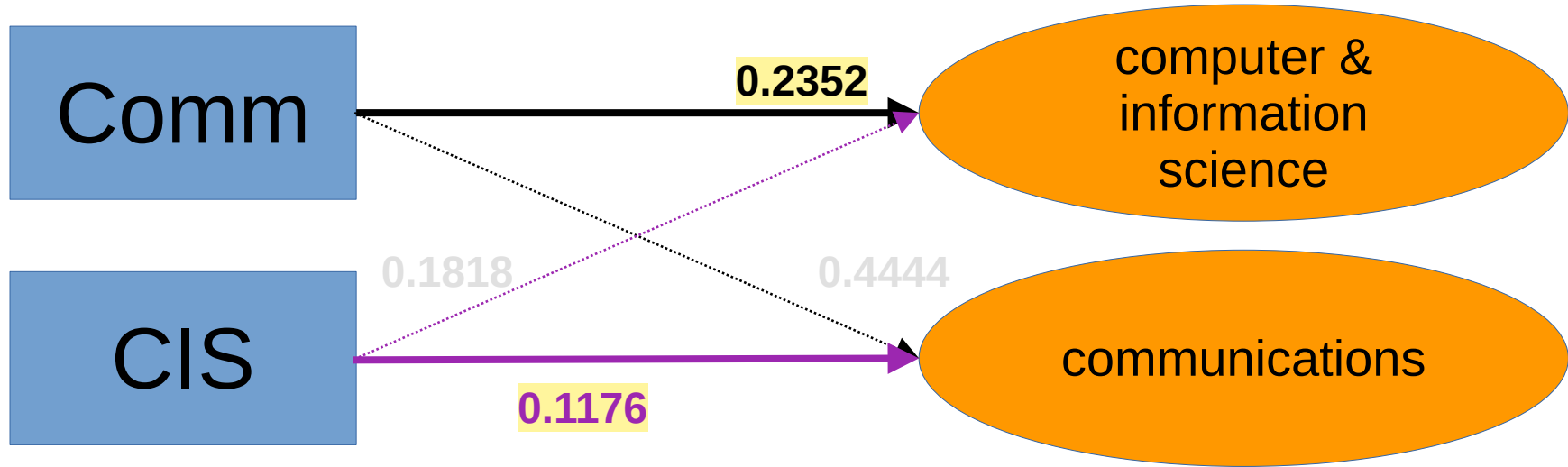
Trick: think **globally**!

Edit Distance



Problem: “comm” is a better match for both!

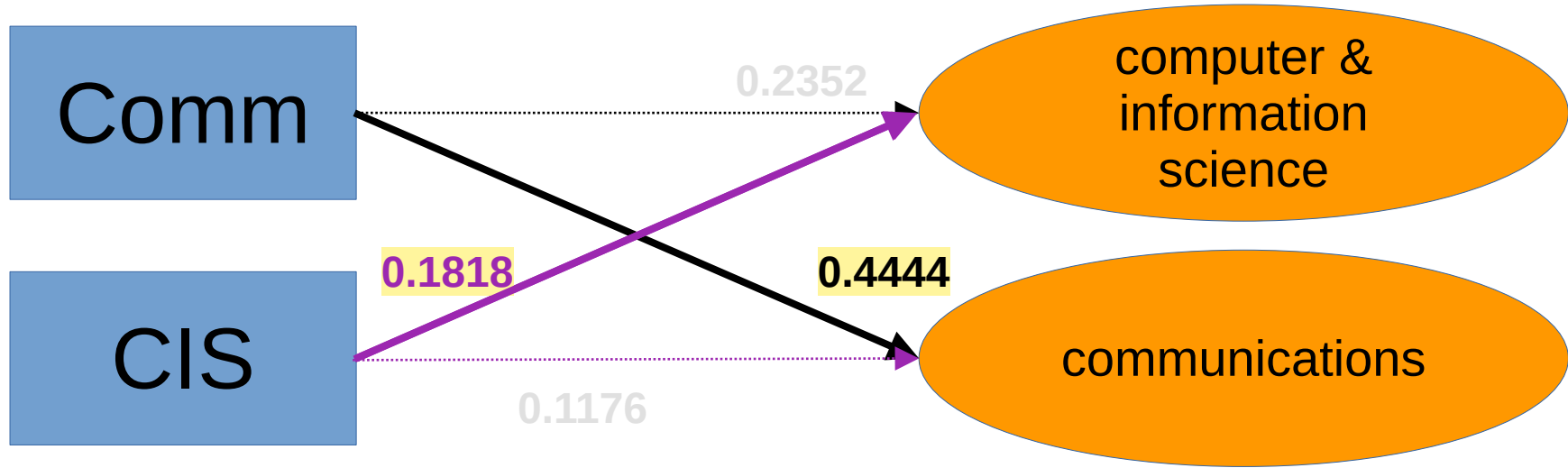
Edit Distance



Try one matching

Score: $0.2352 + 0.1176 = 0.3528$

Edit Distance



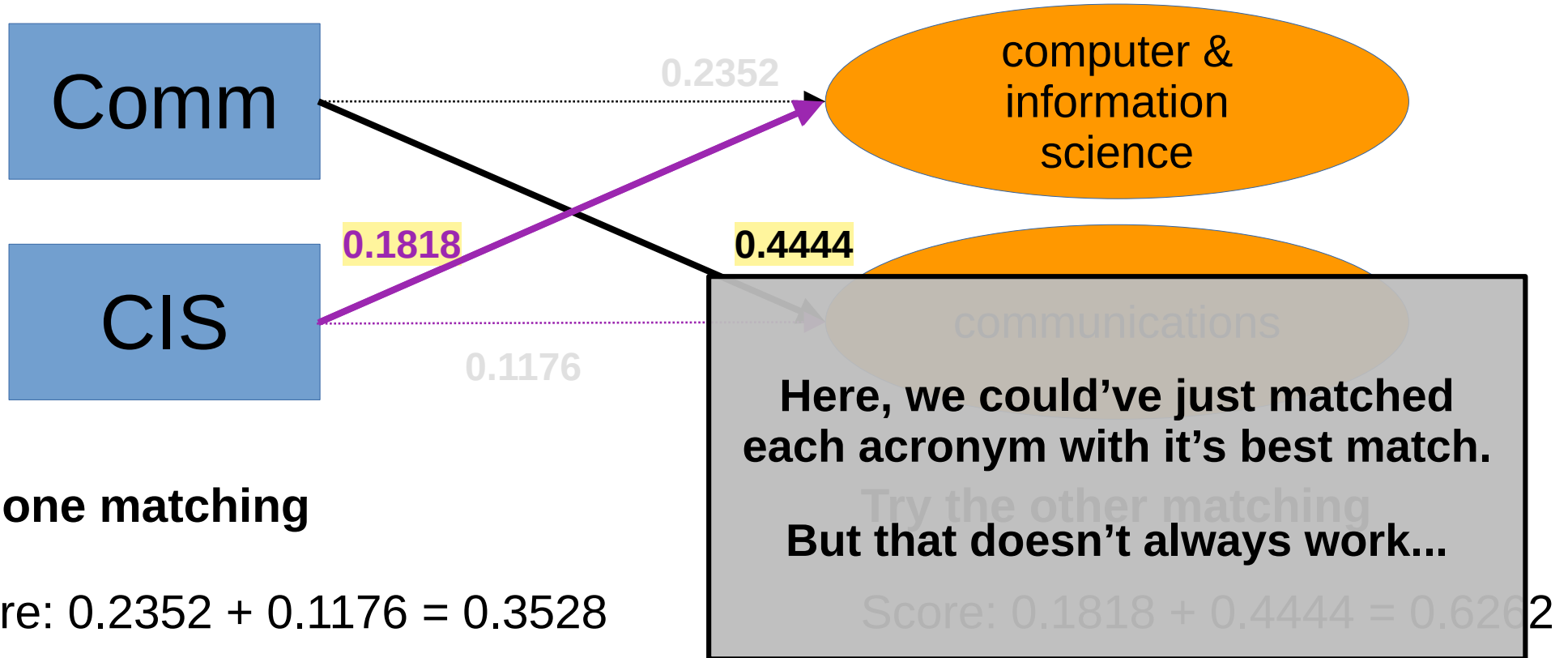
Try one matching

Score: $0.2352 + 0.1176 = 0.3528$

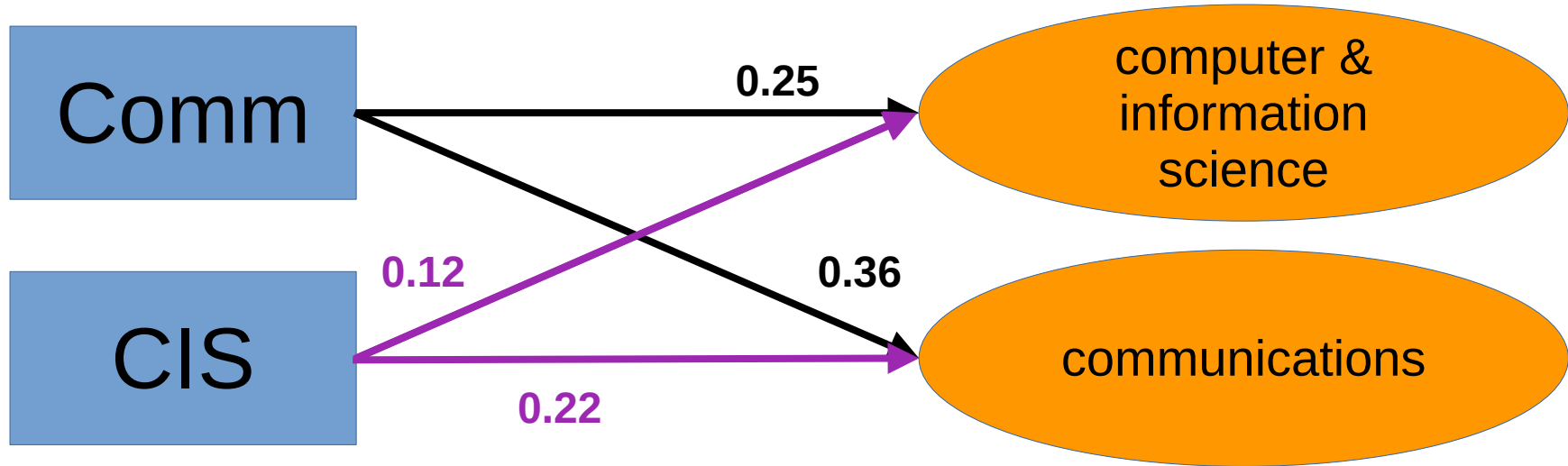
Try the other matching

Score: $0.1818 + 0.4444 = 0.6262$

Edit Distance



Edit Distance



Comm with Computer & Information Science: $0.25 + 0.22 = 0.47$

Comm with Communications: $0.36 + 0.12 = 0.48$

Matching

- The problem of finding a **maximum matching** of a **bipartite graph** is $O(n^3)$
 - For non-CS folks: not very fast
- Let's see it...

df

↔

Acronym

Department

0

AAMW

Art & Arch of Med. World

1

ACCT

Accounting

2

ACFD

Academic Foundations

3

AFRC

Africana Studies

4

ALAN

Asian Languages

...

...

...

209

WOLF

Wolof

210

WRIT

Writing Program

211

YDSH

Yiddish

212

YORB

Yoruba

213

ZULU

Zulu

214 rows × 2 columns

Let's look at Penn **department names** and **acronyms**

We'll start with the ground truth, even though we generally don't know that.

We will *pretend* we don't know which acronym goes with each department.

```
all_to_all = (df.merge(df, how="cross")
              .drop(columns=["Acronym_y", "Department_x"])
              .rename(columns={"Acronym_x": "Acronym",
                              "Department_y": "Department"}))
all_to_all["score"] = all_to_all.apply(lambda row: s(row["Acronym"], row["Department"]), axis=1)
all_to_all
```

Get all pairs of (acro, dept)

Apply the score function to each pair

Drop redundant columns and rename

	Acronym	Department	score
0	AAMW	Art & Arch of Med. World	0.285714
1	AAMW	Accounting	0.142857
2	AAMW	Academic Foundations	0.083333
3	AAMW	Africana Studies	0.100000
4	AAMW	Asian Languages	0.105263
...	...	...	...
45791	ZULU	Wolof	0.000000
45792	ZULU	Writing Program	0.000000
45793	ZULU	Yiddish	0.000000
45794	ZULU	Yoruba	0.000000
45795	ZULU	Zulu	0.250000

45796 rows × 3 columns

Looks promising!

Approach #1: Edit distance

```
[13] all_to_all[all_to_all["Acronym"] == "CIS"].sort_values("score", ascending=False).head(10)
```



	Acronym	Department	score
8445	CIS	Igbo	0.285714
8396	CIS	Czech	0.250000
8384	CIS	Cinema	0.222222
8532	CIS	Swedish	0.200000
8531	CIS	Swahili	0.200000
8390	CIS	College	0.200000
8389	CIS	Classical Studies	0.200000
8382	CIS	Chinese	0.200000
8527	CIS	Spanish	0.200000
8455	CIS	Italian	0.200000



Consider top 10 matches
for CIS

Well-known bias of edit
distance score func



```
guess_with_each_max = all_to_all.loc[all_to_all.groupby("Acronym").idxmax(numeric_only=True)["score"]]  
guess_with_each_max
```



	Acronym	Department	score
0	AAMW	Art & Arch of Med. World	0.285714
409	ACCT	Twi	0.285714
478	ACFD	Czech	0.222222
692	AFRC	Czech	0.222222
973	ALAN	Law	0.285714
...	...	...	...
44843	WOLF	Law	0.285714
45135	WRIT	Twi	0.285714
45206	YDSH	Data Science	0.250000
45580	YORB	Yoruba	0.200000
45699	ZULU	Law	0.285714

214 rows × 3 columns



Try the naive approach:
best match for each

Case-sensitivity combiners
with bias!

In fact, this approach only
gets **17 / 214** correct!

Approach #1: Edit distance



```
import networkx as nx
G = nx.Graph()
G.add_weighted_edges_from(all_to_all.iteruples(index=False, name=None))
results = nx.max_weight_matching(G)
results = [(a, b) if a.isupper() else (b, a) for (a, b) in results]
results
```

Package for graph analysis

Construct bipartite graph

Find the maximum matching

Put acronym first



```
('MAPP', 'Master of Applied Positive Psyc'),
('FILP', 'Indonesian'),
('COLL', 'Latin'),
('INTS', 'International Studies'),
('THAR', 'Tibetan'),
('YORB', 'Yoruba'),
('ETHC', 'Hebrew'),
('BDS', 'Behavioral & Decision Sciences'),
('MTR', 'Thai'),
('CZCH', 'Zulu'),
('LGST', 'Global Studies'),
('MTHS', 'Health & Societies'),
('FNAR', 'Fine Arts'),
('ANCH', 'Classics'),
('ANEL', 'Ancient Near East Languages'),
('MODM', 'Organizational Dynamics'),
('GRMN', 'Management'),
('IMPA', 'Persian'),
('JWST', 'Statistics'),
('WH', 'Hindi'),
('CIMS', 'Cinema and Media Studies'),
```

This approach gets **77 / 214** correct!

Approach #1: Edit distance

```
[27] from fastembed import TextEmbedding
import numpy as np
```

Package for text embeddings

```
embedding_model = TextEmbedding()
embed_cis = next(embedding_model.embed(["CIS"]))
print(embed_cis.shape)
embed_cis
```



Fetching 5 files: 100%  5/5 [00:00<00:00, 67.16it/s]

```
(384,)
array([-5.09407930e-03, -1.65876496e-04,  2.45617796e-02, -5.33128753e-02,
       -4.16888744e-02,  1.19560156e-02,  8.04681703e-02,  5.00113778e-02,
        2.79285293e-02,  1.03112673e-02,  1.58392231e-03, -1.02185912e-01,
        5.73240444e-02,  3.63370702e-02,  4.25936915e-02,  4.85971663e-03,
        2.09923647e-02,  8.04119483e-02, -8.08756519e-03,  2.62987297e-02,
        6.46924600e-02, -4.43695262e-02, -1.73371173e-02, -1.99461170e-02,
       -4.49214242e-02, -6.74803741e-03, -4.27491628e-02, -3.07319704e-02,
       -4.97204512e-02, -9.68213528e-02, -1.41970599e-02, -6.70448244e-02,
        4.30432670e-02,  3.14786918e-02,  2.02608723e-02, -3.14104035e-02,
        1.13618262e-02, -1.70896074e-03, -5.26991896e-02,  1.35574788e-02,
       -4.51088436e-02, -2.51553375e-02, -8.51089433e-02,  2.99567427e-03,
        1.22990645e-02, -3.40101868e-02, -2.47415323e-02,  3.53315519e-03])
```

Transform text into a vector

384 dimensions or
“features”

```
▶ embedding_model = TextEmbedding()
embed_cis = next(embedding_model.embed(["CIS"]))
embed_cis_dept = next(embedding_model.embed(["Computer & Information Science"]))
embed_comm = next(embedding_model.embed(["COMM"]))
embed_comm_dept = next(embedding_model.embed(["Communications"]))

print("CIS -> Comp Sci:", np.dot(embed_cis, embed_cis_dept))
print("CIS -> Communications:", np.dot(embed_cis, embed_comm_dept))
print("COMM -> Communications:", np.dot(embed_comm, embed_comm_dept))
print("COMM -> Comp Sci:", np.dot(embed_comm, embed_cis_dept))
```



Fetching 5 files: 100%  5/5 [00:00<00:00, ...]

```
CIS -> Comp Sci: 0.63693523
CIS -> Communications: 0.622416
COMM -> Communications: 0.68254983
COMM -> Comp Sci: 0.6405632
```

Use dot product to get similarity

COMM is a better match for both, again

```
[95] embedding_model = TextEmbedding()
```

```
    embedded_acs = np.array(list(embedding_model.embed(df["Acronym"])))  
    embedded_depts = np.array(list(embedding_model.embed(df["Department"])))
```



Fetching 5 files: 100%  5/5 [00:00<00:00,

Compute all
embeddings

```
[96] embedded_acs.shape
```



(214, 384)

```
[97] embedded_depts.shape
```



(214, 384)

Note the sizes



```
r = embedded_acs @ embedded_depts.T  
r.shape
```



(214, 214)

Compute pairwise dot
products

$r[i][j]$ is now the score of the i th acronym to the j th department

Approach #2: Embeddings



```
import networkx as nx
G = nx.Graph()

for i in range(r.shape[0]):
    for j in range(r.shape[1]):
        G.add_edge(df["Acronym"][i], df["Department"][j], weight=r[i, j])

results = nx.max_weight_matching(G)
results = [(a, b) if a.isupper() else (b, a) for (a, b) in results]
results
```

Construct bipartite
graph



Find the maximum
matching



```
('PHYL', 'Philosophy'),
('MODM', 'Gujarati'),
('TWI', 'Twi'),
('FOLK', 'Folklore'),
('MSCI', 'Master of Liberal Arts Program'),
('VCSP', 'Vet School Ind Study & Research'),
('THAI', 'Thai'),
('TIGR', 'Tigrinya'),
('ARCH', 'Architecture'),
('CIT', 'Near Eastern Languages & Civlzt'),
('ICOM', 'International Relations'),
('ETHC', 'Ethics')
```

This approach gets **91 / 214**
correct!

```
import lotus
from lotus.models import LM
lotus.settings.configure(
    lm=LM(model="openai/gpt-5.6-luna")
)
```

Configure LOTUS to
use an LLM



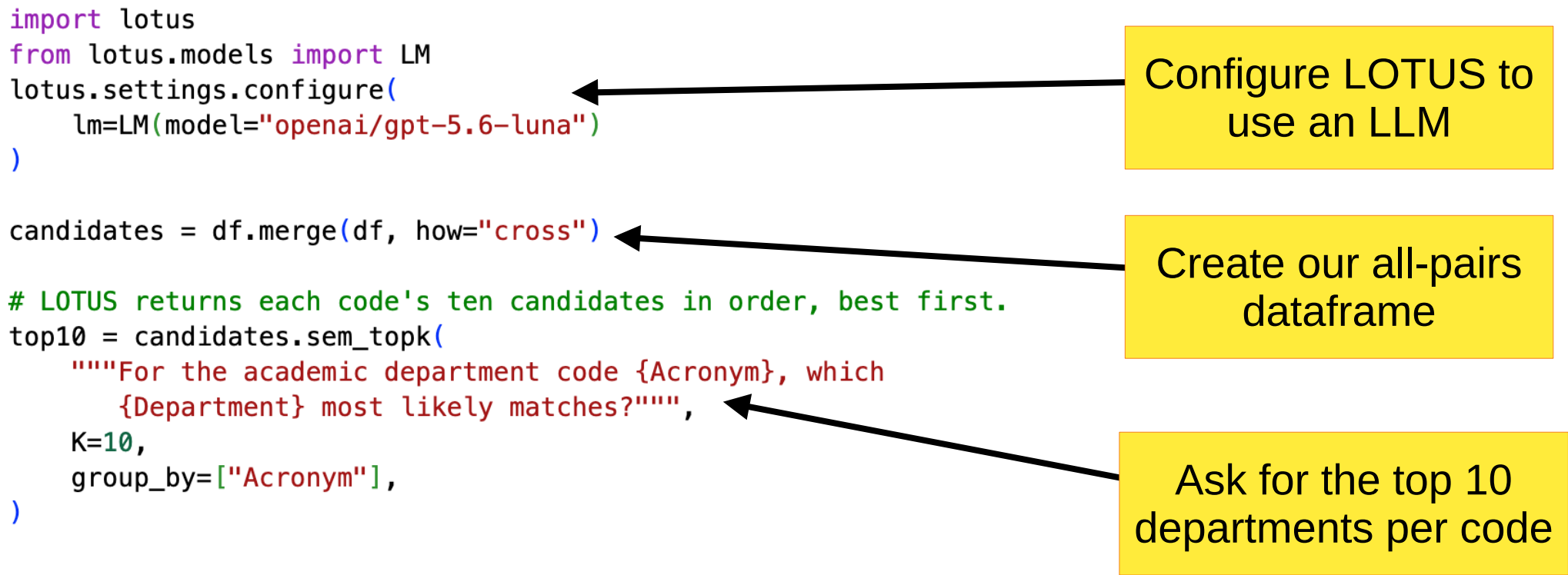
```
candidates = df.merge(df, how="cross")
```

Create our all-pairs
dataframe



```
import lotus
from lotus.models import LM
lotus.settings.configure(
    lm=LM(model="openai/gpt-5.6-luna")
)
```

Configure LOTUS to
use an LLM



```
candidates = df.merge(df, how="cross")
```

Create our all-pairs
dataframe

```
# LOTUS returns each code's ten candidates in order, best first.
```

```
top10 = candidates.sem_topk(
    """For the academic department code {Acronym}, which
    {Department} most likely matches?""",
    K=10,
    group_by=["Acronym"],
)
```

Ask for the top 10
departments per code

```
import lotus
from lotus.models import LM
lotus.settings.configure(
    lm=LM(model="openai/gpt-5.6-luna")
)
```

Configure LOTUS to
use an LLM

```
candidates = df.merge(df, how="cross")
```

Create our all-pairs
dataframe

```
# LOTUS returns each code's ten candidates in order, best first.
```

```
top10 = candidates.sem_topk(
    """For the academic department code {Acronym}, which
    {Department} most likely matches?""",
    K=10,
    group_by=["Acronym"],
)
```

Ask for the top 10
departments per code

Do matching

```
import networkx as nx
G = nx.Graph()
```

```
# Rank 1 earns 10 points, rank 10 earns 1; unranked pairs earn 0.
```

```
for code, matches in top10.groupby("Acronym"):
    for rank, department in enumerate(matches["Department"], start=1):
        G.add_edge(code, department, weight=11 - rank)
results = nx.max_weight_matching(G)
```

Approach #3: LOTUS

```
import lotus
from lotus.models import LM
lotus.settings.configure(
    lm=LM(model="openai/gpt-5.6-luna")
)
```

Configure LOTUS to
use an LLM

```
candidates = df.merge(df, how="cross")
```

Create our all-pairs
dataframe

```
# LOTUS returns each code's ten candidates in order, best first.
top10 = candidates.sem_topk(
    """For the academic department code {Acronym}, which
    {Department} most likely matches?""",
    K=10,
    group_by=["Acronym"],
)
```

Ask for the top 10
departments per code

Do matching

```
import networkx as nx
G = nx.Graph()
```

```
# Rank 1 earns 10 points, rank 10 earns 1; unranked pairs earn 0.
for code, matches in top10.groupby("Acronym"):
    for rank, department in enumerate(matches["Department"], start=1):
        G.add_edge(code, department, weight=11 - rank)
results = nx.max_weight_matching(G)
```

This approach gets
212 / 214 correct!*

Approach #3: LOTUS

Running Time

Method	Accuracy	Time
Best-match	17 / 214	< 1s
Maximum match	77 / 214	10s
Embed + max match	91 / 214	1m40s
LOTUS	212 / 214*	57m, \$12

Is this... good? Bad?

Normally use **human-in-the-loop** techniques

Extra Materials

⋮  05 - Joins & DI.pdf

⋮  (Video) Zack's lecture on entity linking (11m)

⋮  (Video) Outer joins and more in Pandas (14m)

⋮  (Textbook) Intermediate SQL (16 pages)

⋮  (Link) LOTUS Python Library for Semantic Operators